Assignment 5

# Concurrency Tuning

## Database Tuning

B15
Kitajima Mitsuhiro, 12314106
Lastname2 Firstname2, StudentID2
Lastname3 Firstname3, StudentID3

June 9, 2026

### Notes

- You will need to run transactions concurrently using threads in Java. See `https://dbresearch.uni-salzburg.at/teaching/2020ss/dbt/account.zip` for an example.

### Experimental Setup

Describe your experimental setup in a few lines.

[Your answer goes here ...]

### Solution (a)

**Read Committed** Throughput and correctness for solution (a) with isolation level `READ COMMITTED`.

| #Concurrent Transactions | Throughput [transactions/sec] | Correctness |
|---|---|---|
| 1 | 61.63 | 1 |
| 2 | 112.49 | 0.5 |
| 3 | 166.12 | 0.34 |
| 4 | 212.62 | 0.25 |
| 5 | 240.95 | 0.38 |

**Serializable** Throughput and correctness for solution (a) with isolation level `SERIALIZABLE`.

| #Concurrent Transactions | Throughput [transactions/sec] | Correctness |
|---|---|---|
| 1 | 63.48 | 1 |
| 2 | 113.10 | 1 |
| 3 | 155.55 | 1 |
| 4 | 175.17 | 1 |
| 5 | 143.41 | 1 |

### Solution (b)

**Read Committed** Throughput and correctness for solution (b) with isolation level `READ COMMITTED`.

| #Concurrent Transactions | Throughput [transactions/sec] | Correctness |
|---|---|---|
| 1 | 54.09 | 1 |
| 2 | 127.69 | 1 |
| 3 | 171.92 | 1 |
| 4 | 210.28 | 1 |
| 5 | 252.79 | 1 |

**Serializable** Throughput and correctness for solution (b) with isolation level `SERIALIZABLE`.

| #Concurrent Transactions | Throughput [transactions/sec] | Correctness |
|---|---|---|
| 1 | 66.49 | 1 |
| 2 | 105.31 | 1 |
| 3 | 140.40 | 1 |
| 4 | 168.73 | 1 |
| 5 | 143.01 | 1 |

## Discussion

Discuss the outcomes and explain the difference between the isolation levels in PostgreSQL with respect to your experiment.

Explain **with your own words** how PostgreSQL deals with updates in the different isolation levels, within a transaction and within a single SQL command. Explicitly explain why you got the experimental results for solution (a) and (b), respectively.

### Solution (a)

In solution (a), the transactions read the balance into local variables using a `SELECT` query, and then update the balance in a separate `UPDATE` query.

- **Read Committed**: In PostgreSQL, each SQL statement in a transaction runs with a new snapshot of the database. When concurrent transactions run, they can read the same starting balance (e.g., 100) into their local variables. When they execute the update, they overwrite each other's changes using their stale variables. Although row-level locks prevent concurrent updates on the same row, once the lock is released, the blocked transaction proceeds to overwrite the row with its own computed value. This leads to the lost update anomaly, which is why correctness is low (e.g., 0.25–0.50).
- **Serializable**: Under this isolation level, a transaction-level snapshot is used, and PostgreSQL tracks write dependencies. If a transaction attempts to modify a row that has been updated by another transaction since its snapshot started, the transaction is aborted with a serialization failure. Because our testing harness catches this exception, rolls back, and retries the transaction, no updates are lost (correctness is 1.0). However, the overhead from aborts and retries degrades throughput as concurrency increases.

### Solution (b)

In solution (b), the transaction updates the balance directly within a single SQL command.

- **Read Committed**: If a row is currently being updated by a concurrent transaction, a transaction executing an `UPDATE` statement blocks until the active transaction commits or aborts. If it commits, the blocked transaction re-evaluates the query's `WHERE` clause on the new version of the row. Since the clause still matches and the update expression is `balance = balance - 1`, it evaluates using the new committed value of the row. This prevents the lost update anomaly, yielding 1.0 correctness and high throughput (as no transactions need to abort).
- **Serializable**: Similar to solution (a), concurrent updates on the same row (`account = 0`) trigger serialization failures, aborting conflicting transactions. The application's retry loop ensures correctness is 1.0, but the serialization conflict overhead results in lower throughput compared to Read Committed.

## Time Spent on this Assignment

Time in hours per person: **XXX**

## References

---

**Important:** Reference your information sources!
Remove this section if you use footnotes to reference your information sources.

---